

Observability in Kubernetes

Kubernetes Month
Week 3

Disclaimer!

This is meant for Kubernetes Month, not observability in General!

Table of Index

1	What Is Observability, and Why It's More Than Monitoring
2	The Three Pillars of Observability
3	Why Observability Is Critical in Kubernetes
4	Kubernetes Native Observability Signals
5	Metrics, Logs, and Traces in Kubernetes
6	The Observability Stack Best For Kubernetes
7	Prometheus for Kubernetes
8	Grafana for Visualization & Alerting in Kubernetes
9	From Monitoring to Observability Maturity in Kubernetes
10	Recap & What's Next

01

What Is Observability

What Is Observability, and Why It's More Than Monitoring

Think of monitoring as a car dashboard. It tells you the speed, fuel level, and whether a warning light is on. Useful, but when the engine starts making a weird noise, the dashboard alone won't tell you why.

Observability is like being able to open the hood, listen to the engine, and trace where the sound is coming from, all while the car is still running.

In Kubernetes, where applications are spread across many pods and services that constantly appear and disappear, observability helps us understand what is happening, why it is happening, and where to look next. It turns “something is broken” into “this service is slow because this dependency is failing.”

Monitoring



Observability



Monitoring shows symptoms. Observability explains causes.

02

The Three Pillars of Observability

The Three Pillars



2) Logs

Logs are the detailed notes written by the staff; who ordered what, what went wrong, and what actions were taken.

1) Metrics

Imagine you're trying to understand what's happening inside a busy restaurant. Metrics tell you how the restaurant is doing overall, how many customers are inside, how long orders take, and whether the kitchen is overloaded

3) Tracing

Traces follow a single customer from the moment they enter the restaurant until they leave, showing every stop along the way.

On Their Own,

Each pillar gives you part of the story, but together they explain the full experience.

In Kubernetes, these three pillars work the same way, helping you see the system's health, understand events, and follow requests across multiple services.

03

Why Observability Is Critical in Kubernetes

Why Observability Is Critical in Kubernetes

Kubernetes is powerful, but it also hides a lot of complexity behind automation. Pods are constantly created and destroyed, services scale up and down automatically, and workloads move between nodes without warning.

Why Observability Is Critical in Kubernetes

When something breaks, there is often no server to SSH into and no single place where the problem lives. Without observability, failures feel random: a request is slow, a pod restarts, or users complain while everything looks healthy.

Why Observability Is Critical in Kubernetes

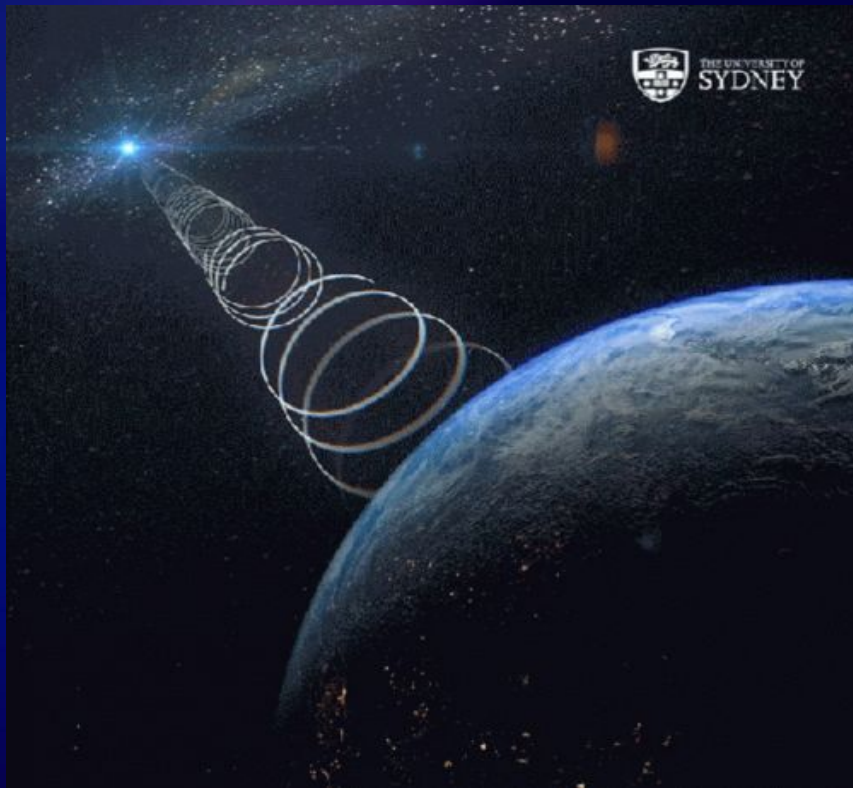
Observability gives visibility into this dynamic environment by showing how applications behave over time, how services depend on each other, and where bottlenecks actually occur. In Kubernetes, observability is not a “nice to have”, it’s the only way to understand what the cluster is really doing.

Kubernetes hides complexity → observability reveals it.

04

Kubernetes Native Observability Signals

Kubernetes Native Observability Signals



Kubernetes is constantly talking, even if you haven't installed a single observability tool yet.

It tells you when a pod starts, stops, crashes, or keeps restarting. It shows how busy each node is and whether it's running out of CPU or memory.

It also explains why things happen through events, like "this pod couldn't start" or "this container was killed." Think of these signals as Kubernetes leaving breadcrumbs about what's going on inside the cluster.

Observability tools don't invent new information; they simply collect, organize, and help you understand what Kubernetes is already telling you.

05

Metrics, Logs, and Traces in Kubernetes



0%

5.1 Metrics in Kubernetes

Metrics answer one simple question:

“How is the system doing right now?”

In Kubernetes, metrics are numbers that go up and down over time, CPU usage, memory consumption, request count, error rate, or response time.

They help you quickly spot patterns, like a pod slowly using more memory, a node getting overloaded, or traffic suddenly spiking.

Metrics are great for answering “Is something wrong?” and “When did it start?”, but not always “Why?”. In Kubernetes, metrics are the first signal that something feels off, and they usually trigger the investigation before you dive deeper into logs or traces.



5.2 Logs in Kubernetes

Logs explaining the problem.

Logs are where Kubernetes applications explain themselves. While metrics tell you something feels wrong, logs tell you what actually happened.

In Kubernetes, applications don't write logs to files you manually open, they write to standard output, and Kubernetes collects them for you.

Every pod, container, and service leaves behind messages like "started successfully," "failed to connect," or "retrying request." Because pods can disappear at any time, logs are often the only record of what went wrong. Logs are especially useful when debugging errors, crashes, or unexpected behavior, because they provide context and detail that numbers alone cannot.



5.3 Traces in Kubernetes

Traces answer the question:

“Where did the request go?”

In Kubernetes, a single user request often travels through multiple services, pods, and containers before a response is returned. Tracing follows that one request step by step, showing how long each service took and where delays or failures occurred.

Instead of guessing which service is slow, traces make the path visible. They are especially useful when everything looks healthy in metrics and logs, but users still experience slowness.

Traces help you see the full journey of a request across the cluster, making distributed systems much easier to understand.

06

The Observability Stack in Kubernetes

The Observability Stack Best For Kubernetes

Think of Kubernetes observability like running a busy airport.

Metrics

Tell you the overall status; how many flights are delayed, and whether traffic is increasing.

In Kubernetes, tools like Prometheus collect these numbers so you can quickly see trends and spot problems

Logs

The detailed reports from the ground crew, what failed, what retried, and what succeeded. Tools such as Loki gather these logs without slowing the system down

Traces

Follow a single passenger from check-in to boarding to landing, showing exactly where delays happen; this is where tools like Tempo shine.

Sitting above everything, Grafana ties it all together, turning raw signals into dashboards, alerts, and stories humans can understand.

07

Prometheus in Kubernetes

Lab Part 1

o8

Grafana for Visualization & Alerting

Lab Part 2

09



From Monitoring to Observability Maturity

From Monitoring to Observability Maturity

Level 1 – Basic Metrics

CPU, memory, and uptime.

You know when something is wrong, but not why.

Level 2 – Logs + Dashboards

You can investigate issues visually
and understand what happened.

From Monitoring to Observability Maturity

Level 3 – Traces + Correlation

You follow requests across services and connect metrics, logs, and traces

Level 4 – Proactive & SLO-Driven

You focus on user experience, detect issues early, and prevent incidents

10

Recap & What's Next

Recap

What We Covered

- What observability means in Kubernetes
- Metrics, logs, and traces (at a high level)
- Prometheus for collecting metrics
- Grafana for visualizing cluster behavior
- How Kubernetes changes the way we observe systems

What's Next

Week 4: Kubernetes for Business

- Why companies adopt Kubernetes
- Cost, reliability, and scalability trade-offs
- Observability as a business enabler
- How technical decisions impact business outcomes

Thanks!

Do you have any questions?

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)